C++ Programming Textbook Notes

Detailed Guide: Comments & All Control Instructions in C++

1. Comments in C++

Conceptual Explanation

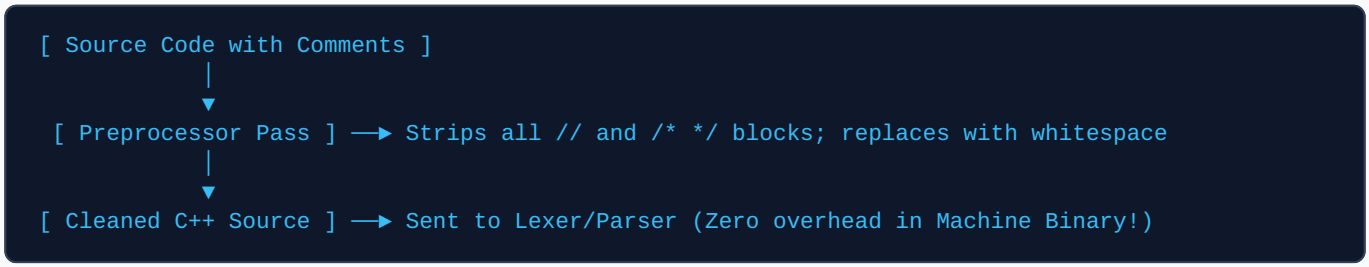
When writing production software, source code is read by human developers far more often than it is compiled. **Comments** are non-executable text annotations written inside source files to explain code intent, document complex mathematical logic, state design assumptions, or store metadata.

During lexical analysis (the initial compiler phase), the preprocessor replaces every comment block with plain whitespace. Consequently, comments consume **0 bytes of RAM** in the output executable, produce zero machine instructions, and have zero impact on runtime performance.

Types of Comments Table

Comment Type	Syntax Marker	Standard Purpose	Example
Single-Line	`//`	Short, inline explanations placed above or beside a code statement.	`int age = 18; // Default voting age`
Multi-Line / Block	`/* ... */`	Multi-paragraph documentation, header licenses, or temporarily disabling code blocks.	`/* Matrix Multiply * Author: Dev */`

Text-Based Diagram: Compiler Preprocessor Comment Removal



Code Example

```cpp
#include <iostream>

/*
 * Program Name: Voting Eligibility Demonstration
 * Author: Computer Science Dept
 * Description: Demonstrates preprocessor comment stripping and conditional logic.
 */
int main() {
    // Define minimum legal age threshold
    const int minVotingAge = 18;
    int userAge = 20; // User input value

    if (userAge >= minVotingAge) {
        std::cout << "Eligible to vote!
";
    }
```

```
    return 0;
}
```

⚠️ **Nesting Block Comments Bug:** Attempting to nest multi-line comments like `/* outer /* inner */ outer */` causes a compilation error because the first `*/` closes the entire block prematurely.

---

2. Introduction to Control Instructions & Truth Evaluation

Conceptual Explanation

By default, execution flows **sequentially**—line by line from top to bottom. **Control Instructions** are keywords and structures that alter this linear CPU instruction pointer, enabling conditional execution branching, looping iterations, and jump conditions.

The Four Categories of Control Instructions

1. **Decision Control Instructions (Selection):** Branching based on true/false conditions (`if`, `if-else`, `if-else-if` ladder).
2. **Iterative Control Instructions (Loops):** Repeating code blocks (`while`, `do-while`, `for`).
3. **Switch-Case Control Instructions:** Multi-way jump selection based on integral matches.
4. **Unconditional Jump Control Instructions:** Non-conditional execution redirection (`goto`, `break`, `continue`, `return`).

Truth in C++: The Golden Evaluation Rule

Modern C++ provides a native `bool` data type with literal constants `true` and `false`. When numeric values or pointer addresses are evaluated inside conditional contexts, C++ enforces the Golden Rule of Truth:

- **TRUE:** Any non-zero value (positive or negative: `1`, `-5`, `100`, non-null pointers).
- **FALSE:** Value of exactly zero (`0`, `0.0`, `nullptr`, `false`).

---

3. Decision Control Instructions (Selection)

A. The `if` Statement

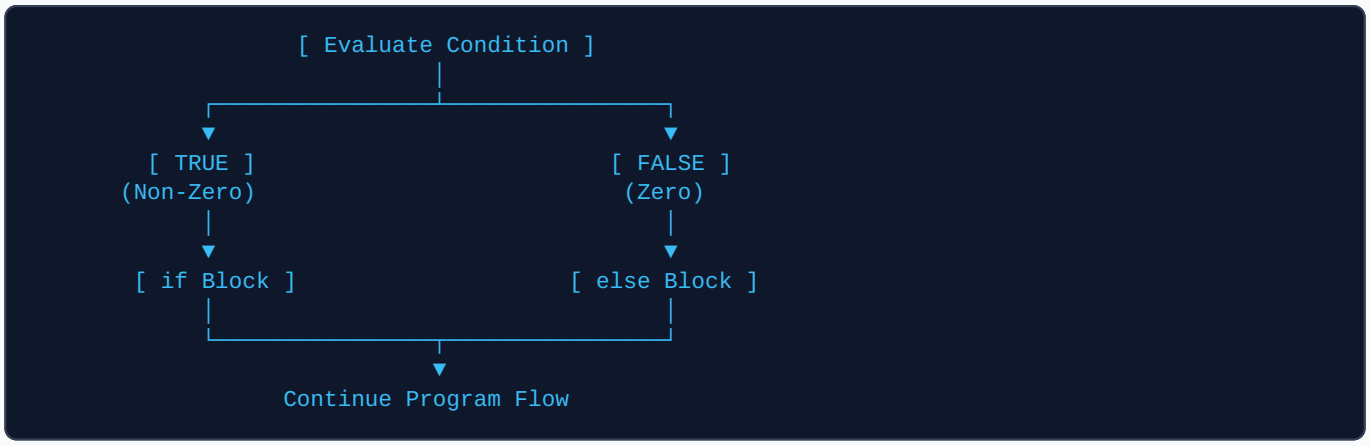
The simplest decision structure. Evaluates a condition; if `true` (non-zero), the inner block runs. If `false` (zero), the compiler skips the block.

```
if (condition) {
    // Executes only if condition evaluates to True (non-zero)
}
```

B. The `if-else` Statement

Provides a mutually exclusive alternate execution branch when the `if` condition fails.

Text-Based Flowchart: If-Else Execution Branching



C. The `if-else-if` Ladder & Ternary Alternate

Tests multiple sequential conditions until one evaluates to `true`. For single inline choices, the ternary operator (`cond ? true_expr : false_expr`) provides a concise alternative.

Detailed Code Example: Positive / Non-Positive & Range Checker

```cpp
#include <iostream>

int main() {
    int number;
    std::cout << "Enter an integer: ";
    std::cin >> number;

    // 1. Standard if-else-if Ladder
    if (number > 0) {
        std::cout << "Result: Positive Number
";
    } else if (number < 0) {
        std::cout << "Result: Negative Number
";
    } else {
        std::cout << "Result: Zero
";
    }

    // 2. Concise Ternary Alternative
    std::cout << "Ternary Check: "
              << ((number > 0) ? "Positive" : "Non-Positive")
              << "
";

    return 0;
}
```

⚠️ **The Assignment vs Comparison Bug:** Writing `if (x = 0)` assigns 0 to x, evaluating to `false` regardless of x's previous value! Writing `if (x = 5)` assigns 5 and evaluates to `true`. Always use double equals (`if (x == 5)`) for equality checks.

> 💡 **Single-Line Rule:** If an `if` or `else` block contains only one statement, curly braces `{}` are technically optional. However, always inclusion of `{}` is a senior C++ best practice to prevent dangling-else bugs during code refactoring.

---

4. Iterative Control Instructions (Loops)

Conceptual Explanation

Loops repeat a code block as long as a controlling boolean condition remains `true`. Every well-designed loop requires three lifecycle steps:

1. **Initialization:** Setting up loop counter tracking variables before iteration starts.
2. **Condition Test:** Evaluating whether the loop body should execute.
3. **Update / Step:** Modifying counter variables towards a terminating state.

Loop Structural Comparison

Loop Type	Control Category	Condition Check Location	Minimum Iteration Count
`while`	Entry-Controlled	Top (Before running body)	**0 Times** (If initially false, body never executes)
`do-while`	Exit-Controlled	Bottom (After running body)	**1 Time** (Guaranteed to execute at least once)
`for`	Counter-Driven	Top (Consolidates init/test/update)	**0 Times**

A. `while` Loop Mechanics & The Infinite Loop Danger

The `while` loop tests its condition at entry. If the state update step inside the loop body is omitted, the condition remains permanently true, resulting in an **Infinite Loop** that locks execution.

```
Infinite Loop Bug Analysis:
  int i = 10;
  while (i) {           ◀── 'i' is 10 (Non-zero = TRUE)
      std::cout << i;  ◀── Prints 10 repeatedly
      // MISSING: i--;  ◀── 'i' remains 10 forever! (App hangs)
  }
```

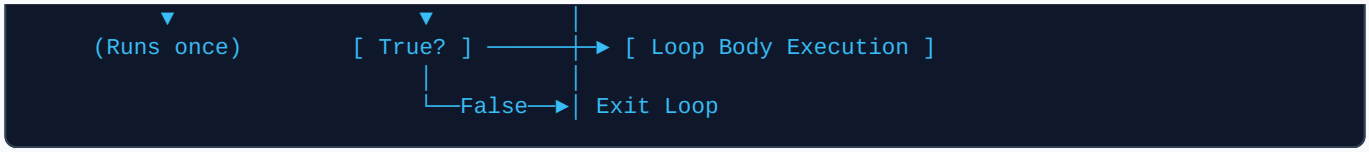
B. `do-while` Loop Mechanics

The `do-while` loop executes its body first, then checks the condition at the bottom.

```
do {
    // Loop body (Runs at least once!)
} while (condition); // MANDATORY trailing semicolon!
```

C. `for` Loop Mechanics & Flow Visualizer

```
for ( initialization ; condition ; update )
        |                  |         ▲
```



Comprehensive Loops Code Example

```cpp
#include <iostream>

int main() {
    // 1. Correct Entry-Controlled while Loop
    int i = 3;
    std::cout << "while loop countdown: ";
    while (i > 0) {
        std::cout << i << " ";
        i--; // CRITICAL: Decrement update step avoids infinite loop
    }

    // 2. Exit-Controlled do-while Loop
    int choice = 0;
    do {
        std::cout << "
Running menu action once minimum...";
    } while (choice != 0); // False immediately, but ran 1 time

    // 3. Counter-Driven for Loop
    std::cout << "
for loop step: ";
    for (int count = 1; count <= 4; count++) {
        std::cout << count << " ";
    }
    std::cout << "
";

    return 0;
}
```

> ⚠️ **Semicolon Trap on Loops:** Placing an accidental semicolon after a `while` or `for` statement (e.g., `while (i > 0); { i--; }`) creates an empty loop body. The program enters an immediate infinite freeze loop!

---

5. Loop Interrupts & Unconditional Jumps (`break`, `continue`, `goto`)

Conceptual Explanation

Flow control interrupts allow abrupt redirection of execution based on specific runtime events:

- `break`: Instantly breaks out of the enclosing loop or `switch` block, jumping execution directly to the statement immediately following the structure.
- `continue`: Skips the remaining statements in the current iteration and jumps directly to the loop condition test / update step.
- `goto`: Performs an unconditional jump to a named statement label within the same function scope.

Text-Based Diagram: `break` vs `continue` Behavior

```
Loop Cycle Execution (e.g., for loop i = 1 to 10):
  [ Iterate Body ] ──► If condition triggers 'continue' ──► Jump straight to update (i++)
  [ Iterate Body ] ──► If condition triggers 'break'    ──► Exit loop immediately! ──► [ Outer Code ]
```

Code Example

```cpp
#include <iostream>

int main() {
    std::cout << "Demonstrating continue and break:
";
    for (int n = 1; n <= 10; n++) {
        if (n % 2 == 0) {
            continue; // Skip printing even numbers
        }
        if (n == 7) {
            break; // Abort loop completely when n reaches 7
        }
        std::cout << "Odd Number: " << n << "
";
    }

    // goto demonstration
    int status = 404;
    if (status == 404) {
        goto errorHandler;
    }

    std::cout << "This line will be skipped
";

errorHandler:
    std::cout << "Handled error code 404 via goto label.
";

    return 0;
}
```

⚠️ **The Danger of `goto`:** Indiscriminate use of `goto` creates "spaghetti code"—unstructured execution paths that obscure logic flow, bypass object destructors, and cause severe software debugging nightmares.

💡 **Best Practice:** Restrict `goto` usage entirely. Use structured loop conditions, `break`, or function returns instead.

---

6. Switch-Case Control Instructions

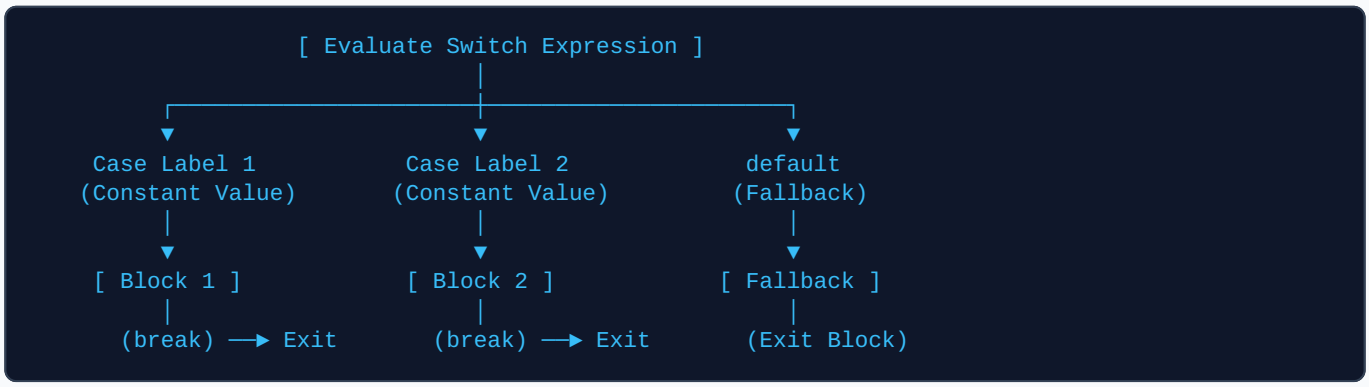
Conceptual Explanation

The `switch` statement acts as an efficient multi-way branch selector. Instead of evaluating a sequence of linear conditions like an `if-else-if` ladder, the compiler often optimizes a `switch` block using a **Jump Table** (direct memory branch offset based on index values).

Strict Rules for `switch` in C++

1. **Integral / Enumeration Types Only:** The switch expression must evaluate to an integer (`int`, `char`, `short`, `long`) or an `enum class`. Floating-point types (`float`, `double`) are strictly illegal!
2. **Constant Case Labels:** Case values must be compile-time constants or raw literals (e.g., `case 5:` or `case 'A':`). Variables (e.g., `case x:`) cause compilation errors.
3. **Unique Labels:** Every case constant in a single `switch` block must be strictly unique. Duplicate labels trigger immediate compilation errors.
4. **Fall-Through Mechanics:** Execution flows sequentially from a matched case into subsequent cases unless halted explicitly by a `break` statement.
5. **The `default` Block:** An optional fallback block executed when no explicit case constant matches.

Text-Based Diagram: Switch Jump Table Branching



Code Example

```cpp
#include <iostream>

int main() {
    char userGrade = 'B';

    switch (userGrade) { // Must evaluate to integral type (char is integer ASCII)
        case 'A':
            std::cout << "Grade A: Score >= 90%
";
            break; // Stops fall-through execution
        case 'B':
            std::cout << "Grade B: Score >= 80%
";
            break;
        case 'C':
            std::cout << "Grade C: Score >= 70%
";
            break;
        default: // Fallback choice
            std::cout << "Grade below C or invalid input.
";
            break;
    }

    return 0;
}
```

⚠️ **Unintended Fall-Through Bug:** Forgetting a `break` statement at the end of a case block causes execution to cascade straight into subsequent case blocks, executing unexpected code.

💡 **C++17 `[[fallthrough]]` Attribute:** If intentional fall-through is required by design, C++17 provides the `[[fallthrough]];` attribute annotation to suppress compiler warnings.